Université de Strasbourg
Master Mathématiques et Applications – Parcours Statistique

Identification individuelle et analyse du réseau social de macaques japonais par apprentissage profond

Projet AI-SARU – Population de Koshima

Prénom NOM

Année universitaire 2025–2026

Remerciements

Table des matières

Introduction **3**

1 Méthodes de détection automatique par apprentissage profond **4**
1.1 Introduction à l'apprentissage supervisé et à l'apprentissage profond 4
1.2 Détection d'objets et principe général du modèle YOLO 6
1.3 Évaluation statistique des performances du modèle 8
1.4 Application du modèle YOLO au projet 9

2 Données et prétraitement **10**
2.1 Présentation et acquisition des données 10
2.2 Organisation et préparation des données 11
2.3 Nettoyage et annotation des images 11
2.4 Correction et validation des détections 12

3 Construction des matrices de proximité sociale **14**
3.1 Détection des individus et évaluation des résultats 14
3.2 Extraction des coordonnées et calcul des distances 14
3.3 Construction de la matrice de proximité sociale à partir des détections 15
3.4 Construction de la matrice de référence à partir des observations de terrain . . . 15

4 Comparaison statistique des matrices **17**
4.1 Méthodologie de comparaison des matrices 17
4.2 Présentation théorique du test de Mantel 17
4.3 Application du test de Mantel et résultats 17
4.4 Interprétation des résultats 18

5 Discussion et perspectives **19**
5.1 Discussion des résultats 19
5.2 Limites et conditions d'application des méthodes 19
5.3 Perspectives d'amélioration 19

Conclusion générale **20**

Bibliographie **20**

A Annexes **21**
A.1 Extraits de code 21
A.2 Figures complémentaires 21
A.3 Tableaux complémentaires 21

Introduction

Contexte scientifique et présentation du projet

Le stage s'est déroulé à l'Institut Pluridisciplinaire Hubert Curien (IPHC), unité mixte de recherche du CNRS et de l'Université de Strasbourg (UMR 7178), au sein du Département d'Écologie, Physiologie et Éthologie (DEPE). Ce département étudie les stratégies adaptatives mises en œuvre par les espèces animales pour faire face aux contraintes de leur environnement, à travers des approches d'écologie comportementale et d'écophysiologie, tant fonctionnelles qu'évolutives.

C'est dans ce cadre que s'inscrit le projet AI-SARU, dont l'objectif est de transformer des enregistrements vidéo bruts de populations de macaques japonais en données éthologiques directement exploitables scientifiquement, via une chaîne de traitement automatisée combinant détection des comportements, reconnaissance individuelle des animaux, et mesure de leur proximité sociale. L'automatisation de ces tâches, traditionnellement réalisées par observation humaine directe sur le terrain, répond à un besoin méthodologique bien identifié dans la littérature : Schofield *et al.* [**?**] ont notamment démontré, sur une population de chimpanzés sauvages suivie pendant 14 ans, qu'un pipeline entièrement automatisé de détection, suivi et reconnaissance faciale par apprentissage profond permet d'atteindre une précision d'identification supérieure à 92%, tout en traitant des volumes de données (plusieurs millions de détections) hors de portée d'un traitement manuel. Leurs travaux illustrent également l'intérêt de regrouper les détections en *tracks* avant de statuer sur l'identité d'un individu plutôt que de décider image par image, une approche directement reprise dans le pipeline AI-SARU (cf. chapitre 3).

Contexte du stage et problématique

Objectifs du stage

Organisation du rapport

“‘latex

Chapitre 1

Méthodes de détection automatique par apprentissage profond

Ce chapitre présente les principaux outils mathématiques et statistiques nécessaires à la compréhension du modèle utilisé dans le cadre du projet. L'objectif n'est pas de décrire exhaustivement l'architecture de YOLO, mais d'expliquer les mécanismes qui permettent de localiser un visage dans une image, de lui attribuer une identité et d'évaluer la qualité des prédictions obtenues.

1.1 Introduction à l'apprentissage supervisé et à l'apprentissage profond

Cadre de l'apprentissage supervisé. On se place dans le cadre de l'apprentissage supervisé. On dispose d'un échantillon d'apprentissage

$$\mathcal{D}_n = \{(X_i, Y_i)\}_{i=1}^n \,, \tag{1.1}$$

où X_i désigne une image et où $Y_i \in \{1, \ldots, K\}$ représente l'identité connue de l'individu présent dans cette image, parmi les K individus de la population étudiée.

L'objectif est de construire une fonction de prédiction $\widehat{f}_\theta$, dépendant d'un ensemble de paramètres θ, telle que

$$\widehat{Y}_i = \widehat{f}_\theta(X_i) \tag{1.2}$$

soit aussi proche que possible de l'identité réelle Y_i.

Il s'agit d'un problème de classification supervisée multiclasse, puisque la variable à prédire est qualitative et peut prendre plus de deux modalités. Cette situation se distingue d'un problème de régression, dans lequel la variable réponse est quantitative.

Sortie probabiliste du modèle. Pour une image X_i, le réseau produit généralement un vecteur de scores qui est transformé en probabilités estimées :

$$\widehat{\boldsymbol{p}}_\theta(X_i) = (\widehat{p}_{\theta,1}(X_i), \ldots, \widehat{p}_{\theta,K}(X_i)) \,, \tag{1.3}$$

avec

$$\widehat{p}_{\theta,k}(X_i) \geq 0, \qquad \sum_{k=1}^{K} \widehat{p}_{\theta,k}(X_i) = 1. \tag{1.4}$$

L'identité prédite est alors celle dont la probabilité estimée est la plus élevée :

$$\widehat{Y}_i = \underset{k\in\{1,\dots,K\}}{\arg\max}\ \widehat{p}_{\theta,k}(X_i). \tag{1.5}$$

Ces probabilités permettent non seulement de fournir une prédiction, mais également de quantifier le degré de confiance associé aux différentes identités candidates.

Fonction de perte et maximum de vraisemblance. Dans un problème de classification multiclasse, une fonction de perte couramment utilisée est l'entropie croisée. Elle s'écrit

$$\mathcal{L}_{\text{cls}}(\theta) = -\frac{1}{n}\sum_{i=1}^{n} \log\left(\widehat{p}_{\theta,Y_i}(X_i)\right). \tag{1.6}$$

Cette fonction pénalise fortement les situations dans lesquelles le modèle attribue une faible probabilité à la classe réelle.

D'un point de vue statistique, son utilisation peut être reliée au principe du maximum de vraisemblance. Sous l'hypothèse que les observations sont indépendantes conditionnellement aux paramètres du modèle, la vraisemblance s'écrit

$$\mathcal{V}(\theta) = \prod_{i=1}^{n} \widehat{p}_{\theta,Y_i}(X_i). \tag{1.7}$$

La log-vraisemblance est donc

$$\log \mathcal{V}(\theta) = \sum_{i=1}^{n} \log\left(\widehat{p}_{\theta,Y_i}(X_i)\right). \tag{1.8}$$

Minimiser l'entropie croisée définie dans l'équation (1.6) revient ainsi à maximiser la log-vraisemblance, à un facteur multiplicatif négatif près. L'apprentissage du classifieur peut donc être interprété comme une estimation des paramètres par maximum de vraisemblance.

Réseaux de neurones convolutifs. Les images possèdent une structure spatiale : deux pixels voisins ont généralement davantage de relations qu'une paire de pixels très éloignés. Les réseaux de neurones convolutifs, ou CNN, exploitent cette structure à l'aide de filtres locaux.

Pour une image d'entrée X et un noyau W de dimension $q \times q$, une carte de caractéristiques peut être définie par

$$F(i,j) = \sum_{u=1}^{q}\sum_{v=1}^{q} W(u,v)X(i+u, j+v) + b, \tag{1.9}$$

où b est un biais.

En pratique, l'opération réalisée dans de nombreuses bibliothèques d'apprentissage profond correspond techniquement à une corrélation croisée, mais elle est habituellement appelée convolution dans la littérature sur les CNN.

Le résultat est ensuite transformé par une fonction d'activation non linéaire. Une fonction courante est la fonction ReLU :

$$\phi(x) = \max(0, x). \tag{1.10}$$

L'empilement de plusieurs couches permet au réseau d'apprendre des représentations de complexité croissante. Les premières couches extraient généralement des motifs locaux simples, comme les contours ou les textures, tandis que les couches plus profondes combinent ces informations pour former des représentations plus discriminantes.

Estimation des paramètres. Les paramètres du réseau sont estimés en recherchant une valeur $\widehat{\theta}$ minimisant la fonction de perte :

$$\widehat{\theta} = \arg\min_{\theta} \mathcal{L}(\theta). \tag{1.11}$$

Cette optimisation ne possède généralement pas de solution analytique, en raison du grand nombre de paramètres et de la non-convexité de la fonction de perte. Les paramètres sont donc mis à jour de manière itérative par une méthode fondée sur le gradient :

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_{\theta} \mathcal{L}\left(\theta^{(t)}\right), \tag{1.12}$$

où $\eta > 0$ est le taux d'apprentissage.

En pratique, le gradient est généralement estimé sur un sous-échantillon d'observations appelé *mini-batch*. Cette procédure conduit à la descente de gradient stochastique ou à l'une de ses variantes, telles qu'Adam. La rétropropagation permet ensuite de calculer efficacement les dérivées de la fonction de perte par rapport à l'ensemble des paramètres du réseau.

Dans le cadre d'un ré-entraînement ou *fine-tuning*, l'optimisation ne commence pas à partir de paramètres aléatoires. Elle s'appuie sur les paramètres d'un modèle préalablement entraîné, puis les adapte aux nouvelles données. Dans le projet étudié, le principe consiste à exploiter un modèle déjà ajusté sur une autre population de macaques, puis à le réadapter aux individus de la population de Koshima. Cette description devra toutefois être précisée à partir des informations exactes fournies par l'encadrant concernant le protocole d'entraînement effectivement utilisé.

1.2 Détection d'objets et principe général du modèle YOLO

Le modèle employé dans le projet appartient à la famille YOLO, acronyme de *You Only Look Once* [**?**]. Les modèles de cette famille sont conçus pour effectuer une détection d'objets directement à partir d'une image complète.

Contrairement à un classifieur recevant une image déjà recadrée, un détecteur doit résoudre simultanément plusieurs tâches :

— déterminer si un objet d'intérêt est présent ;
— localiser cet objet à l'aide d'une boîte englobante ;
— attribuer une classe à l'objet localisé.

Dans le cadre du projet, la localisation permet d'obtenir les coordonnées des visages détectés, tandis que la classification permet d'associer chaque visage à une identité candidate.

Représentation d'une boîte englobante. Une boîte englobante peut être représentée par son centre et ses dimensions :

$$B = (c_x, c_y, w, h), \tag{1.13}$$

où (c_x, c_y) désigne le centre de la boîte, w sa largeur et h sa hauteur.

Elle peut également être décrite par les coordonnées de ses coins :

$$B = (x_{\min}, y_{\min}, x_{\max}, y_{\max}). \tag{1.14}$$

Ces informations sont particulièrement importantes dans ce projet, car les coordonnées des boîtes seront ensuite utilisées pour calculer une mesure de distance entre les individus détectés.

Prédiction en une seule étape. Le principe général de YOLO consiste à produire, lors d'une même évaluation du réseau, les prédictions de localisation, de présence d'objet et de classification. Cette approche se distingue des anciennes méthodes en deux étapes, qui généraient d'abord des régions candidates avant de les classifier séparément.

Le traitement conjoint de l'image rend cette famille de modèles adaptée à l'analyse de vidéos comportant un nombre important d'images successives.

Les détails précis de l'architecture, notamment le recours à une grille, à des boîtes d'ancrage ou à une représentation sans ancres, dépendent de la version de YOLO utilisée. Ils ne doivent donc être décrits dans le rapport qu'après identification exacte de la version et du paramétrage employés dans le projet.

Intersection sur union. La qualité géométrique d'une boîte prédite peut être évaluée à l'aide de l'intersection sur union, notée IoU. Pour une boîte prédite B_{pred} et une boîte de référence B_{ref}, elle est définie par

$$\text{IoU}\left(B_{\text{pred}}, B_{\text{ref}}\right) = \frac{\text{aire}\left(B_{\text{pred}} \cap B_{\text{ref}}\right)}{\text{aire}\left(B_{\text{pred}} \cup B_{\text{ref}}\right)}. \tag{1.15}$$

Cette quantité appartient à l'intervalle $[0, 1]$. Une valeur proche de 1 indique un fort recouvrement entre la boîte prédite et la boîte de référence, tandis qu'une valeur proche de 0 correspond à un faible recouvrement.

Dans une procédure d'évaluation, une détection est généralement considérée comme correctement localisée lorsque son IoU dépasse un seuil fixé. La valeur exacte de ce seuil doit être précisée si elle a été utilisée dans le projet.

Scores de confiance et filtrage des prédictions. Chaque détection est associée à un ou plusieurs scores représentant notamment la confiance du modèle dans la présence d'un objet et dans son appartenance à une classe.

Après la prédiction, un seuil de confiance peut être appliqué afin d'écarter les détections jugées trop incertaines. Un traitement supplémentaire est également nécessaire lorsque plusieurs boîtes se recouvrent fortement autour d'un même objet.

La suppression non maximale, ou NMS, consiste à conserver la boîte ayant le score le plus élevé, puis à supprimer les autres boîtes dont le recouvrement avec celle-ci dépasse un seuil donné. Cette procédure réduit le nombre de détections redondantes.

Fonction de perte d'un détecteur. L'entraînement d'un détecteur d'objets repose généralement sur une fonction de perte composée de plusieurs termes :

$$\mathcal{L}_{\text{det}} = \lambda_{\text{loc}}\mathcal{L}_{\text{loc}} + \lambda_{\text{obj}}\mathcal{L}_{\text{obj}} + \lambda_{\text{cls}}\mathcal{L}_{\text{cls}}, \tag{1.16}$$

où :

— $\mathcal{L}_{\text{loc}}$ mesure l'erreur de localisation des boîtes englobantes ;
— $\mathcal{L}_{\text{obj}}$ mesure l'erreur relative à la présence ou à l'absence d'un objet ;
— $\mathcal{L}_{\text{cls}}$ mesure l'erreur de classification ;
— les coefficients λ_{loc}, λ_{obj} et λ_{cls} règlent l'importance relative de ces composantes.

La forme exacte de chacune de ces composantes dépend de la version de YOLO utilisée. Certaines versions emploient, par exemple, des fonctions de perte fondées sur l'IoU ou sur des variantes telles que la GIoU, la DIoU ou la CIoU pour évaluer la localisation des boîtes.

Il serait donc incorrect de présenter une fonction de perte détaillée comme étant celle du modèle du projet sans avoir vérifié la version exacte du réseau. L'équation (1.16) fournit ici une représentation générale du principe d'optimisation.

1.3 Évaluation statistique des performances du modèle

L'évaluation du modèle doit distinguer deux aspects :
— la qualité de la localisation des visages ;
— la qualité de l'identification individuelle.

La localisation peut être étudiée à l'aide de l'IoU et de métriques propres à la détection d'objets. L'identification individuelle peut être évaluée à partir d'une matrice de confusion construite en comparant les identités prédites aux identités réelles.

Matrice de confusion. Pour un problème comportant K individus, la matrice de confusion est une matrice $C = (C_{kl})_{1 \leq k,l \leq K}$ dans laquelle

$$C_{kl} = \text{nombre d'observations d'identité réelle } k \text{ prédites comme } l. \tag{1.17}$$

Les valeurs diagonales C_{kk} correspondent aux identifications correctes, tandis que les valeurs hors diagonale décrivent les erreurs de classification.

La matrice permet notamment d'identifier les couples d'individus fréquemment confondus par le modèle.

Précision et rappel. Pour un individu k, la précision est définie par

$$\text{Prcision}_k = \frac{C_{kk}}{\sum_{l=1}^{K} C_{lk}}. \tag{1.18}$$

Elle mesure la proportion de prédictions associées à l'identité k qui sont effectivement correctes.

Le rappel est défini par

$$\text{Rappel}_k = \frac{C_{kk}}{\sum_{l=1}^{K} C_{kl}}. \tag{1.19}$$

Il mesure la proportion d'observations réellement associées à l'individu k qui sont correctement reconnues.

F1-score. Le F1-score est la moyenne harmonique de la précision et du rappel :

$$F1_k = 2\frac{\text{Prcision}_k \, \text{Rappel}_k}{\text{Prcision}_k + \text{Rappel}_k}. \tag{1.20}$$

Cette métrique est utile lorsqu'un compromis entre précision et rappel est recherché. Elle doit toutefois être interprétée avec précaution lorsque certains individus sont beaucoup plus représentés que d'autres.

Exactitude globale. Le taux d'exactitude global, ou *accuracy*, est défini par

$$\text{Exactitude} = \frac{\sum_{k=1}^{K} C_{kk}}{\sum_{k=1}^{K}\sum_{l=1}^{K} C_{kl}}. \tag{1.21}$$

Il représente la proportion totale de prédictions correctes.

Cependant, lorsque les effectifs diffèrent fortement entre individus, cette mesure peut être dominée par les classes les plus représentées. Il est donc préférable de compléter l'exactitude globale par des métriques calculées individu par individu, puis éventuellement par leurs moyennes macro :

$$\text{Rappel}_{\text{macro}} = \frac{1}{K} \sum_{k=1}^{K} \text{Rappel}_k \,. \tag{1.22}$$

Cette moyenne accorde le même poids à chaque individu, indépendamment de son nombre d'observations.

Ensemble d'évaluation. Afin que les métriques mesurent réellement la capacité de généralisation du modèle, elles devraient idéalement être calculées sur des données n'ayant pas servi à son entraînement.

Si les mêmes images sont utilisées pour entraîner puis évaluer le réseau, les performances risquent d'être surestimées. La constitution de l'échantillon de vérité terrain et sa relation avec les données d'entraînement devront donc être décrites précisément dans le chapitre 2.

L'application de ces métriques aux résultats obtenus pendant le stage sera présentée au chapitre 3, dans la section 3.1.

1.4 Application du modèle YOLO au projet

Le choix d'un modèle de détection de la famille YOLO a été effectué par l'encadrant du stage avant le début de celui-ci. Ce choix s'appuyait notamment sur des résultats déjà obtenus sur une autre population de macaques.

Son utilisation répond à deux besoins complémentaires du projet.

Premièrement, la localisation des visages permet d'extraire les coordonnées des boîtes englobantes. Ces coordonnées constituent les informations nécessaires au calcul ultérieur des distances spatiales entre individus.

Deuxièmement, la classification permet d'associer chaque boîte englobante à une identité. Sans cette étape, il serait possible de mesurer la distance entre deux détections, mais pas d'attribuer cette proximité à une paire précise de macaques.

La chaîne méthodologique peut ainsi être résumée par

$$\text{image} \longrightarrow \text{détection} \longrightarrow \text{localisation} \longrightarrow \text{identification} \longrightarrow \text{mesure de proximité.} \tag{1.23}$$

Le traitement conjoint de la localisation et de l'identification rend cette approche adaptée à l'exploitation de vidéos comportant un grand nombre d'images.

Les prédictions obtenues ne sont toutefois pas considérées comme automatiquement exactes. Elles doivent être contrôlées, corrigées et évaluées avant d'être utilisées dans la construction d'une matrice de proximité sociale. La nature des données, leur organisation et les opérations de prétraitement sont présentées dans le chapitre suivant. “‘

Chapitre 2

Données et prétraitement

Ce chapitre présente les données utilisées au cours du stage ainsi que les différentes étapes permettant de passer des enregistrements vidéo bruts à un jeu d'images annotées et exploitable par les modèles de détection et d'identification. Une attention particulière est portée à la sélection des images, à la correction manuelle des prédictions et à l'organisation des annotations produites.

2.1 Présentation et acquisition des données

Les données étudiées proviennent d'enregistrements vidéo de macaques japonais appartenant à la population de Koshima, au Japon. Le jeu de données disponible dans le cadre du projet AI-SARU représente environ quinze heures de vidéos et concerne une population d'environ quarante individus.

Ces enregistrements ont été collectés sur le terrain afin de documenter les comportements et les relations sociales entre les individus. La vidéo constitue une source d'information particulièrement adaptée à cet objectif, car elle permet d'observer simultanément plusieurs animaux, leurs déplacements et leur position relative au cours du temps.

Cependant, les vidéos brutes ne peuvent pas être utilisées directement dans une analyse statistique. Elles doivent d'abord être transformées en observations structurées comportant notamment l'identité des individus, leurs coordonnées dans l'image et les instants auxquels ils apparaissent. Le projet AI-SARU vise précisément à automatiser cette transformation grâce à une chaîne de traitement combinant détection, identification individuelle et mesure de proximité.

Les fichiers vidéo suivent une convention de nommage de la forme

`AAAAMMJJ_n`,

où `AAAA`, `MM` et `JJ` correspondent respectivement à l'année, au mois et au jour de l'enregistrement, tandis que `n` désigne le numéro de la vidéo enregistrée ce jour-là. Par exemple, le nom `20230302_1` désigne la première vidéo enregistrée le 2 mars 2023.

Les images utilisées pendant le stage étaient extraites automatiquement de ces vidéos au moyen d'un script Python. Le script parcourait chaque vidéo et enregistrait des images à intervalles réguliers. Les dossiers d'images ainsi obtenus constituaient les données d'entrée de l'interface de détection et d'identification.

La fréquence d'extraction n'était pas nécessairement identique pour toutes les vidéos. Lorsqu'un dossier contenait trop peu d'images exploitables, le paramètre contrôlant le nombre d'images extraites par seconde était augmenté. Cette adaptation permettait d'enrichir le jeu de données tout en conservant une couverture temporelle suffisante de la vidéo.

2.2 Organisation et préparation des données

Pour chaque vidéo, les images extraites étaient regroupées dans un dossier portant un nom permettant de conserver le lien avec l'enregistrement d'origine. Les noms des images contenaient également le nom de la vidéo et le numéro de la frame extraite. Un nom tel que

`20230330_9_..._frame_000035.jpg`

permet ainsi d'identifier la vidéo source et la position de l'image dans la séquence.

Après la sélection des images, le dossier correspondant était compressé au format ZIP afin d'être chargé dans l'interface de prédiction. Cette interface, développée dans le cadre du projet, était lancée depuis le terminal à l'aide d'une série de commandes spécifiques.

Trois éléments principaux devaient être fournis lors du lancement :

— le fichier `detector.pt`, correspondant au modèle chargé de détecter les visages de macaques ;
— le fichier `identificator.pt`, correspondant au modèle chargé d'attribuer une identité à chaque visage détecté ;
— le dossier d'images compressé au format ZIP.

Le premier modèle localisait les visages présents dans les images à l'aide de boîtes englobantes. Le second associait ensuite à chaque détection une identité parmi les individus connus de la population de Koshima.

À l'issue du traitement, les résultats étaient enregistrés dans un fichier JSON dont la structure est proche du format COCO. Ce fichier est organisé en trois blocs principaux :

— `images` : une entrée par image, contenant notamment son identifiant, son nom de fichier, sa hauteur et sa largeur ;
— `annotations` : une entrée par détection, contenant l'identifiant de l'image, les coordonnées de la boîte englobante (`bbox`), le score de confiance de la détection (`confidence`), l'identité retenue (`category_id` et `category_name`) et, lorsqu'elles sont disponibles, les informations de suivi temporel telles que `track_id` ;
— `categories` : la liste des identités pouvant être attribuées aux détections.

Une boîte englobante est enregistrée sous la forme

$$\texttt{bbox} = [x, y, w, h], \tag{2.1}$$

où (x, y) représente le coin supérieur gauche de la boîte, w sa largeur et h sa hauteur, en pixels.

Pour chaque détection, le champ `extended_classification` contient également les cinq identités jugées les plus probables par le modèle, notées `top1` à `top5`, ainsi que leurs scores respectifs. Ces informations permettent de visualiser le degré d'incertitude du modèle et de vérifier si l'identité réelle figure parmi les premières propositions.

Dans le fichier fourni à titre d'exemple, les images ont une résolution de 640×360 pixels. Le bloc `categories` contient 42 identités de macaques, numérotées de 0 à 41.

Le fichier JSON comprend par ailleurs un champ `proximity`, initialement vide dans les annotations examinées. Ce champ est destiné à recevoir, dans une étape ultérieure du pipeline, les informations relatives aux individus estimés proches de l'individu détecté. Le calcul de cette proximité est présenté au chapitre 3.

2.3 Nettoyage et annotation des images

Avant leur utilisation dans l'interface, les dossiers d'images faisaient l'objet d'un nettoyage manuel. Cette étape avait pour objectif d'améliorer la qualité et la représentativité du jeu de données utilisé pour l'identification des macaques.

Les images trop floues étaient supprimées lorsque le visage de l'individu ne pouvait pas être identifié de manière suffisamment fiable. Une image de mauvaise qualité peut en effet conduire à une annotation incertaine et introduire du bruit dans le jeu de données d'entraînement.

Les images fortement répétitives étaient également retirées. Lorsque plusieurs frames consécutives représentaient presque exactement la même scène et le même individu sous un angle identique, leur conservation apportait peu d'information nouvelle. Elle pouvait en revanche accentuer artificiellement la représentation de certaines situations ou de certains individus dans le jeu de données.

Le nettoyage devait toutefois rester compatible avec la nécessité de disposer d'un nombre suffisant d'exemples pour chaque individu. Lorsqu'un dossier contenait trop peu d'images après suppression des frames floues ou redondantes, la fréquence d'extraction du script Python était augmentée, puis les images étaient générées de nouveau à partir de la vidéo.

Le dossier nettoyé était ensuite compressé et chargé dans l'interface avec les deux modèles `detector.pt` et `identificator.pt`. L'interface affichait les boîtes englobantes proposées par le détecteur ainsi que l'identité considérée comme la plus probable par le modèle d'identification.

Cette première sortie constituait une préannotation automatique. Elle réduisait le temps nécessaire à l'annotation, mais ne pouvait pas être utilisée sans contrôle humain, car le modèle pouvait omettre un visage, détecter un élément ne correspondant pas à un macaque ou attribuer une identité incorrecte.

2.4 Correction et validation des détections

Chaque prédiction était examinée manuellement dans l'interface. Plusieurs types de corrections pouvaient être réalisés.

Lorsqu'un visage de macaque était visible dans l'image mais n'avait pas été détecté par le modèle, une nouvelle boîte englobante pouvait être ajoutée manuellement autour du visage. Cette situation correspond à une détection manquée par le modèle.

À l'inverse, lorsqu'une boîte englobante ne correspondait pas réellement à un visage de macaque, la fausse détection pouvait être supprimée.

Enfin, lorsque la boîte était correctement positionnée mais que l'identité proposée en première position par le modèle était incorrecte, le nom de l'individu pouvait être modifié manuellement. Les cinq propositions affichées par le modèle facilitaient parfois cette correction lorsque l'identité réelle figurait parmi les propositions secondaires.

Les corrections réalisées dans l'interface étaient ensuite enregistrées. Cette opération générait un fichier `coco_annotations.json` dans un répertoire de travail nommé `workdir`. Le fichier produit conservait les informations relatives aux images, aux boîtes englobantes et aux identités validées.

Le fichier JSON était ensuite déplacé dans le dossier correspondant aux images traitées. Cette organisation permettait de conserver ensemble les données visuelles et leurs annotations, et donc de retrouver facilement les annotations associées à chaque lot d'images.

Avant le traitement d'un nouveau dossier, le contenu du répertoire `workdir` devait être supprimé. Cette précaution évitait de mélanger les fichiers générés lors de deux traitements différents ou d'associer par erreur un fichier JSON au mauvais dossier d'images.

Les fichiers JSON corrigés avaient ainsi deux fonctions principales. D'une part, ils constituaient un jeu d'annotations validées pouvant être réutilisé pour l'entraînement ou le réentraînement des modèles. D'autre part, ils fournissaient les informations nécessaires aux étapes suivantes du pipeline, notamment les identités et les coordonnées des boîtes englobantes utilisées pour calculer la proximité entre individus.

Pour évaluer la qualité des identifications produites par le modèle, les identités prédites pouvaient également être comparées aux identités validées manuellement. Pour chaque observation, l'identité réelle et l'identité prédite étaient reportées dans un tableau de comparaison. Ce ta-

bleau constitue la vérité terrain nécessaire au calcul de la matrice de confusion et des métriques présentées au chapitre 1, section 1.3.

Les résultats numériques de cette évaluation sont présentés au chapitre 3, section 3.1. Les annotations corrigées sont quant à elles utilisées comme données d'entrée du programme Python chargé d'extraire les coordonnées, de consolider les identités et de construire la matrice de proximité sociale.

Chapitre 3

Construction des matrices de proximité sociale

3.1 Détection des individus et évaluation des résultats

Consolidation de l'identité par *track*. Les détections ne sont pas traitées indépendamment image par image : elles sont regroupées en *tracks*, c'est-à-dire des séquences de détections correspondant à un même individu suivi sur plusieurs frames consécutives. Pour un track t, on cumule sur toutes ses frames les scores de confiance associés à chaque identité candidate k :

$$S_t(k) = \sum_{\text{frame } f \in t} \text{score}_f(k). \tag{3.1}$$

L'identité retenue pour le track est celle qui maximise ce score cumulé, sous deux conditions de fiabilité :

— un support relatif suffisant, $\dfrac{\max_k S_t(k)}{\sum_k S_t(k)} \geq \tau_{\text{support}}$ (seuil par défaut : 60%) ;

— un nombre minimal de frames dans le track (par défaut : 3).

Cette consolidation par vote pondéré agrégé rend l'identification plus robuste qu'une décision frame par frame, chaque frame individuelle pouvant être bruitée.

Résultats d'évaluation.

3.2 Extraction des coordonnées et calcul des distances

Pour deux individus a et b détectés dans une même frame, on note $(c_{x,a}, c_{y,a})$ et $(c_{x,b}, c_{y,b})$ les centres de leurs boîtes de détection respectives. La distance centre-centre s'écrit

$$d_{ab} = \sqrt{(c_{x,a} - c_{x,b})^2 + (c_{y,a} - c_{y,b})^2}. \tag{3.2}$$

Cette distance en pixels ne suffit pas à elle seule, car elle ne tient pas compte de la profondeur : un même écart en pixels peut correspondre à une grande distance réelle si les individus sont loin de la caméra, ou une faible distance s'ils en sont proches. La hauteur de la boîte de détection sert de proxy à l'échelle apparente de l'individu, corrigée d'un facteur $\alpha(\text{âge})$ (les individus jeunes, plus petits, sont sous-corrigés sinon) :

$$h^{\text{corr}} = h \times \alpha(\text{âge}). \tag{3.3}$$

La distance normalisée est alors

$$d_{ab}^{\text{norm}} = \frac{d_{ab}}{\max(h_a^{\text{corr}}, h_b^{\text{corr}})}. \tag{3.4}$$

Un ratio de taille des aires corrigées sert de garde-fou contre les incompatibilités de profondeur :

$$r_{ab} = \frac{\max(\text{aire}_a^{\text{corr}}, \text{aire}_b^{\text{corr}})}{\min(\text{aire}_a^{\text{corr}}, \text{aire}_b^{\text{corr}})}. \tag{3.5}$$

Si r_{ab} dépasse un seuil τ_{ratio}, la paire est jugée trop incertaine en profondeur et exclue.

La paire (a, b) est déclarée **proche** dans la frame si

$$d_{ab}^{\text{norm}} \leq \tau_{\text{dist}} \quad \text{et} \quad r_{ab} \leq \tau_{\text{ratio}}. \tag{3.6}$$

3.3 Construction de la matrice de proximité sociale à partir des détections

Agrégation frame vers vidéo. Pour chaque vidéo v et chaque paire (a, b) co-détectée, on calcule la proportion de frames où la paire est proche :

$$\text{score}_v(a, b) = \frac{\text{nombre de frames "proches"}}{\text{nombre de frames co-détectées valides}}. \tag{3.7}$$

Agrégation vidéo vers matrice globale. Les frames d'une même vidéo ne constituent pas des observations indépendantes : deux individus filmés proches pendant 30 secondes ne fournissent pas autant d'information que 900 observations indépendantes de proximité, mais plutôt une unique situation répétée à haute fréquence d'échantillonnage. Pour limiter le poids disproportionné des vidéos longues, chaque vidéo se voit attribuer un poids plafonné :

$$w_v = \min(\text{nombre de frames co-détectées valides}_v,\ w_{\max}), \tag{3.8}$$

avec $w_{\max} = 900$ par défaut (soit 30 secondes à 30 images/seconde).

La matrice de proximité globale $\hat{M}$ est alors une moyenne pondérée des scores vidéo :

$$\hat{M}_{ab} = \frac{\sum_v w_v \cdot \text{score}_v(a, b)}{\sum_v w_v}. \tag{3.9}$$

Propriétés de la matrice obtenue.

3.4 Construction de la matrice de référence à partir des observations de terrain

La matrice de référence M^* est construite à partir d'observations de terrain réalisées directement par des chercheurs, sur la même période temporelle que les vidéos utilisées pour le ré-entraînement du modèle. Contrairement à la matrice estimée $\hat{M}$, qui repose uniquement sur la fréquence de proximité spatiale détectée automatiquement, M^* agrège plusieurs dimensions du lien social observées sur le terrain : la fréquence de coprésence, les interactions de toilettage (*grooming*), les relations de dominance, ainsi que les liens de parenté entre individus. M^*_{ab} traduit ainsi une intensité globale de proximité sociale entre les individus a et b, au sens large.

Cette différence de nature entre les deux matrices doit être nuancée. La matrice estimée $\hat{M}$ ne mesure explicitement qu'une proximité spatiale ; elle ne distingue pas la nature du lien à l'origine de cette proximité. Cependant, les épisodes de grooming ou de jeu, qui impliquent par nature un contact ou une grande proximité physique entre les individus concernés, se traduisent directement par des scores de proximité élevés dans $\hat{M}$: ces comportements sont donc captés *indirectement* par la mesure spatiale, sans être reconnus ou différenciés en tant que tels. $\hat{M}$ ne peut en revanche

pas distinguer une proximité spatiale « fortuite » (par exemple deux individus qui se nourrissent côte à côte sans interaction sociale particulière) d'une proximité liée à un comportement social significatif, ni capter des liens sociaux ne se traduisant pas par une coprésence spatiale fréquente (par exemple un lien de parenté entre deux individus qui se croisent rarement). La matrice de référence M^*, construite à partir d'observations de terrain intégrant explicitement la nature des interactions (grooming, dominance, parenté), permet cette distinction que $\hat{M}$ ne fait pas. Cette asymétrie doit être gardée à l'esprit dans l'interprétation de leur comparaison (cf. section 5.2).

Chapitre 4

Comparaison statistique des matrices

4.1 Méthodologie de comparaison des matrices

Les entrées de la matrice $\hat{M}$ (et de la matrice de référence M^*) ne sont pas indépendantes entre elles : la ligne associée à un individu a particulièrement sociable sera globalement élevée sur toute la ligne, car les valeurs $\hat{M}_{ab}$ et $\hat{M}_{ac}$ partagent la même dépendance à a. Un test de corrélation classique (Pearson), qui suppose l'indépendance des observations, ne peut donc pas être appliqué directement entre les entrées des deux matrices. Une méthode adaptée à cette structure de dépendance est nécessaire : le test de Mantel.

4.2 Présentation théorique du test de Mantel

Le **test de Mantel** [**?**] est spécifiquement conçu pour comparer deux matrices de distance ou de proximité définies sur le même ensemble d'unités (ici, les K individus).

Statistique de test. On considère la corrélation (par exemple de Pearson) entre les entrées des deux matrices, restreinte aux paires distinctes (a, b), $a \neq b$:

$$Z_{\text{obs}} = \text{corr}\big(\{\hat{M}_{ab}\}_{a<b},\ \{M^*_{ab}\}_{a<b}\big). \tag{4.1}$$

Hypothèse nulle et test par permutation. L'hypothèse nulle H_0 postule l'absence d'association entre les deux structures de proximité. Comme les entrées d'une même matrice ne sont pas indépendantes (une ligne entière dépend de la sociabilité globale d'un individu), la distribution théorique usuelle de la statistique de corrélation ne s'applique pas. On construit donc la distribution de Z sous H_0 par **permutation** :

1. on permute aléatoirement l'ordre des individus (lignes et colonnes simultanément) dans l'une des deux matrices, par exemple M^*, donnant M^*_π pour une permutation π ;
2. on recalcule la statistique de corrélation Z_π entre $\hat{M}$ et M^*_π ;
3. on répète l'opération un grand nombre de fois (typiquement 999 ou 9999 permutations), donnant une distribution empirique $\{Z_\pi\}$ sous H_0.

La p-valeur empirique est alors la proportion de permutations pour lesquelles Z_π est au moins aussi extrême que Z_{obs} :

$$p = \frac{1 + \#\{\pi : |Z_\pi| \geq |Z_{\text{obs}}|\}}{1 + \text{nombre de permutations}}. \tag{4.2}$$

4.3 Application du test de Mantel et résultats

Vérification des conditions d'application.

— Les deux matrices doivent être définies sur le même ensemble d'individus, avec le même ordre de lignes/colonnes avant permutation.
— Le nombre de permutations doit être suffisant pour estimer précisément la p-valeur (au minimum quelques centaines).
— Les valeurs manquantes (paires jamais co-détectées dans $\hat{M}$) doivent être traitées explicitement (exclusion de la paire dans les deux matrices, ou imputation à justifier).
— Le test de Mantel suppose que la structure de dépendance interne à chaque matrice est correctement neutralisée par la permutation conjointe des lignes/colonnes – ce qui est le cas ici, la permutation appliquée à un individu déplace simultanément toute sa ligne et sa colonne.

Résultats obtenus.

4.4 Interprétation des résultats

Chapitre 5

Discussion et perspectives

5.1 Discussion des résultats

5.2 Limites et conditions d'application des méthodes

5.3 Perspectives d'amélioration

Conclusion générale

Annexe A

Annexes

A.1 Extraits de code

```
def evaluate_pair_in_frame(a, b, dist_thresh_norm, size_ratio_thresh):
    dist_norm = distance(a, b) / max(a["corrected_h"], b["corrected_h"])
    size_ratio = max(a["aire"], b["aire"]) / min(a["aire"], b["aire"])
    close = (dist_norm <= dist_thresh_norm) and (size_ratio <=
        size_ratio_thresh)
    return close
```

A.2 Figures complémentaires

A.3 Tableaux complémentaires